thyme v0.1.0 · Typst >= 0.15.1

Pure-Typst static site generator with typed hypertagged HTML, Tadi Web .typ source transparency, zero-JS CSS i18n, XML sitemaps & feeds, and a monochrome blog preset.

core ssg	`#import "../src/lib.typ": *`
blog preset	`#import blog: *` or `blog.site.with(...)`
scaffold	`typst init @preview/thyme:0.1.0 my-site`
dev / build	`typst watch --features html,bundle -f bundle site.typ dist`
license	`MIT` (`src/`) · `MIT-0` (`template/`, `examples/`)

1 Two-Layer Architecture & Pure-Typst Workflow 1
2 Tadi Web `.typ` Sources & `sws.toml` Headers 2
3 Core SSG Reference (`src/*.typ`) 2
3.1 Site Bundler, Subpages & Raw Assets (`src/core.typ`) 2
3.2 Typed Hypertagged HTML & Hyperscript (`src/htm.typ`) 4
3.3 Zero-JS Single-URL CSS i18n (`src/i18n.typ`) 5
3.4 Sitemap, `robots.txt` & Feeds (`src/seo.typ`) 7
3.5 URLs, Slugs, AST Traversal & Plain Text (`src/paths.typ`, `src/text.typ`, `src/tree.typ`) 9
4 `blog` Preset Reference (`src/blog/*.typ`) 15
4.1 Blog Bundler (`src/blog/site.typ`) 15
4.2 Post Template (`src/blog/post.typ`) 17
4.3 Scholarly Environments & Components (`src/blog/components.typ`) 20
4.4 Right-Margin Sidenotes (`src/blog/notes.typ`) 24
4.5 Monochrome Theme & Fonts (`src/blog/theme.typ`) 25

01 Two-Layer Architecture & Pure-Typst Workflow

`thyme` separates static-site orchestration from visual theming across two composable layers, driven exclusively by the standard `typst` CLI (`typst compile` and `typst watch`) without external wrapper CLIs or client-side JavaScript runtimes[1]:

[1] Typst `0.15.1`'s `typst watch -f bundle` includes a built-in live-reloading HTTP server (`--port`, `--no-reload`, `--no-serve`).

LAYER	IMPORT	RESPONSIBILITIES
Core SSG	`#import "../src/lib.typ": *`	Unopinionated zero-CSS `site`, `webpage`, `mount(path(...))`, `file`, `backlinks`, `htm` / `h` typed hypertagged HTML, `#tr(...)` zero-JS CSS i18n + `hreflang` alternates, sibling `.typ` source co-emission, `sitemap.xml`, `robots.txt`, OpenGraph/Scholar `<meta>`, and summary or full-HTML `rss.xml` / `atom.xml`.
`blog` Preset	`#import blog: *` / `blog.site`	Opinionated scholarly monochrome blog (`oklch` / `light-dark()` + 2-column `@media print`), *STIX Two* + *Helvetica Neue* + *Commit Mono*, `@preview/marge` sidenotes + `#mcite` margin citations, static `/tags/` archives, `#toc()`, `#post-nav()`, Obsidian `<dl>` frontmatter, theorem/proof boxes, `zebraw`, `tablem`, and `cheq`.

02 Tadi Web `.typ` Sources & `sws.toml` Headers

When pages are mounted via Typst capability paths (`src: path("site.typ")`, `#mount(path("contact.typ"))`, `#post(path("posts/foo.typ"))`), `thyme` reads and co-emits the verbatim `.typ` source alongside each HTML page (`site.typ`, `contact.typ`, `posts/foo.typ`) and links it via `.typ` in the top utility bar. Configure your static server (such as `static-web-server`) so browsers render `.typ` inline as UTF-8 plaintext:

toml

```toml
[[advanced.headers]]
source = "**/*.typ"
headers = { Content-Type = "text/plain; charset=utf-8", Content-Disposition =
"inline" }
```

03 Core SSG Reference (`src/*.typ`)

3.1 Site Bundler, Subpages & Raw Assets (`src/core.typ`)

with-html-math(body) -> content

Normalize Typst HTML/MathML output using pure syntactic AST `show` rules (zero `measure()`, zero SVG, 100% native `<math>`):

1. Suppresses auto-`math.lr` `stretchy="true"` on single-line expressions (`$F(e)$`, `$F(d^*)$`) so Safari/WebKit keeps delimiters on the math axis/baseline while still stretching tall fractions, matrices, roots, big operators (`$F(a/b)$`, `$F(union.sq.big_n d_n)$`), and explicit `lr(...)`.
2. Coerces lone operator glyphs in script slots (`$d^*$`, `$ℝ^+$`, `$G^×$`) to `math.class("normal", ...)` so MathML engines do not pad superscripts/subscripts with binary `lspace`/`rspace`.

PARAMETERS.

body content

Content subtree to normalize for HTML MathML output.

```
site(
  title: auto,
  description: none,
  url: "https://example.com",
  author: none,
  lang: "en",
  alternates: none,
  stylesheets: (),
  favicon: none,
  assets: (),
  src: none,
  expose-source: true,
  not-found-page: true,
  sitemap-path: "sitemap.xml",
  robots: auto,
  rss-path: none,
  atom-path: none,
  feed-content: "summary",
  feed-epoch: auto,
  heading-permalinks: false,
  backlinks: false,
  open-graph: false,
  scholar-meta: false,
  wrap-main: true,
  head: none,
  ..extra,
  body,
) -> content
```

Generic, unopinionated static site bundler (`#show: site.with(...)`).

Compiles a pure-Typst project into a multi-page static HTML bundle (`index.html`, mounted subpages, user stylesheets, static files, `404.html`, `sitemap.xml`, `robots.txt`, optional RSS/Atom feeds, and Tadi Web `.typ` plaintext source files) without any opinionated CSS or JavaScript runtime. Import `blog` and use `blog.site` (or `#import blog: *`) to activate the scholarly monochrome `blog` preset without import-order sensitivity.

PARAMETERS.

`title` `auto` `content` `str` = auto

Site title (`auto` infers from the first `= ...` heading in `body`, or pass any `str` / rich `content`).

`description` `content` `str` `none` = none

Short site description for `<meta name="description">` and feed subtitles.

`url` `str` = "https://example.com"

Canonical site origin URL (without trailing slash) for canonical links, `sitemap.xml`, and feeds.

`author` `content` `str` `none` = none

Default site author.

`lang` `str` `array` = "en"

Default site language (`"en"` or `("en", "de")`).

`alternates` `dictionary` `array` `none` = none

Optional `hreflang` alternate URLs for the site root (`(de: "/de/")`).

`stylesheets` `array` = ()

Stylesheets to emit and link (`path("styles.css")` or `(path: "styles.css", data: ..., href?: ..., label?: ...)`).

`favicon` `path` `dictionary` `none` = none

Optional site favicon (`path("assets/favicon.svg")` or `(path: "favicon.svg", data: ...)`).

`assets` `array` = ()

Additional static files (`(path: "robots.txt", data: ..., label: none)`) copied into the bundle root.

`src` `path` `str` `none` = none

Capability (`path("site.typ")`) (or `str`) of the root entrypoint so Tadi Web source view can emit (`/site.typ`).

`expose-source` `bool` `str` = true

Co-emit sibling (`.typ`) plaintext source files (`/site.typ`, `/<route>.typ`) and render a top-right link (`true` uses `".typ"`, `false` disables, or pass custom label `str`).

`not-found-page` `bool` `content` = true

Emit (`404.html`) (`true` emits a minimal root-relative 404 page, or pass custom `content`).

`sitemap-path` `str` `none` = "sitemap.xml"

Output path for the XML sitemap (`"sitemap.xml"`), or `none` to disable sitemap generation.

`robots` `auto` `bool` = auto

Emit `robots.txt` pointing to `sitemap-path` (`auto` enables whenever `sitemap-path != none`).

`rss-path` `str` `none` = none

Output path for the RSS 2.0 feed (e.g. `"rss.xml"`), or `none` to omit RSS on non-blog sites.

`atom-path` `str` `none` = none

Output path for the Atom 1.0 feed (e.g. `"atom.xml"`), or `none` to omit Atom on non-blog sites.

`feed-content` `str` = "summary"

Feed payload mode (`"summary"` for compact descriptions or `"full"` for full semantic HTML).

`feed-epoch` `auto` `datetime` = auto

Optional RFC 4151 feed `<id>` minting date (`auto` derives from earliest post).

`heading-permalinks` `bool` = false

Append `§` hover permalinks on section headings (`h2`–`h6`) across pages.

`backlinks` `bool` = false

Render compile-time backlinks footers across pages/posts by default.

`open-graph` `bool` = false

Emit OpenGraph `<meta property="og:*">` tags across pages/posts by default.

`scholar-meta` `bool` = false

Emit Google Scholar Highwire `<meta name="citation_*">` tags on articles by default.

`wrap-main` `bool` = true

Wrap the root `index.html` body inside `<main>` (`false` lets `body` emit `<header>`/`<main>`/`<footer>` directly, like `apex`).

`head` `content` `none` = none

Additional `<head>` elements injected on `index.html`.

`..extra` `any`

Extra preset-specific options forwarded when `preset != none` (and internal preset hooks).

`body` `content`

Root homepage markup and `#mount(path(...))` / `#file(...)` declarations.

3.2 Typed Hypertagged HTML & Hyperscript (`src/htm.typ`)

`h`(`selector`, `..args`) -> `content`

Construct an HTML element from a concise CSS-style selector string (`"section#projects.entries[data-tier=2]"`) backed by Typst's typed `html.*` module. When passed a `length`, `relative`, `ratio`, or `fraction` (e.g. `#h(1fr)` or `#h(0.5em)`), transparently delegates to Typst's built-in horizontal spacing function `std.h`.

```
#h("div.callout#intro")[Hello from *hyperscript*.]
```

Hello from **hyperscript**.

PARAMETERS.

`selector` `str` `relative` `fraction`

Tag + `#id` + `.class` + `[attr=val]` selector (`"div"` assumed when tag is omitted), or a spacing amount (`length` / `fraction`) forwarded to `std.h`.

`..args` `any`

Additional named attributes (`href: "..."`, `open: true`), optional attribute dictionaries, and child content blocks (or `weak: bool` when forwarding to `std.h`).

htm(template, ..slots) -> content

Parse Hypertagged Markup (`develop it/htm`-style HTML fragments with `{slot}` Typst content interpolation) directly into Typst's native typed `html.*` elements.

Accepts either a fenced `raw` block (`lang: "html"`) or a `str` template, plus named (`name: [...]`) or positional (`{0}`) Typst content slots.

```
#htm(
  "<section class=\"intro\"><p><strong>{greeting}</strong> — {note}</p></section>",
  greeting: [Grüezi],
  note: [compiled straight to native HTML elements with $O(n)$ parsing.],
)
```

Grüezi — compiled straight to native HTML elements with $O(n)$ parsing.

PARAMETERS.

template content str
HTML fragment (`raw` block or `str`) with optional `{slot}` placeholders (`{{` / `}}` for literal braces).

..slots any
Named (`key: content`) or positional (`{0}`, `{1}`) Typst values spliced into `{slot}` positions.

enable-htm(body) -> content

Optional show-rule helper (`#show: enable-htm`) that compiles `lang: "htm"` raw blocks directly into HTML elements via `htm(...)`.

PARAMETERS.

body content
Document body.

3.3 Zero-JS Single-URL CSS i18n (src/i18n.typ)

```
resolve-page-langs(
  lang,
  default-lang: "en",
  scan-nodes: (),
  queried-codes: (),
) -> array
```

Normalize a `lang` configuration (`"en"`, `("en", "de")`, or `auto`) plus any `#tr(...)` language codes discovered in a page's content tree into a deduplicated array of BCP-47 language strings (primary language first).

PARAMETERS.

lang auto str array
Explicit page `lang` (`auto`, `"en"`, or `("en", "de")`).

default-lang str array = "en"
Fallback primary language from `site.with(lang: ...)`.

scan-nodes array = ()
Content nodes to scan for `#tr(...)` blocks when `lang == auto`.

queried-codes array = ()
Additional language codes discovered via `query(<thyme-tr-meta>)` inside `context`.

normalize-alternates(alternates, site-url: none) -> array

Normalize `alternates` (`("de": "/de/about/", "en": "/about/")` or `((hreflang: "de", href: "..."),)`) into canonical `(hreflang: str, href: str)` records for `<link rel="alternate" hreflang="...">` and `sitemap.xml` `xhtml:link`.

PARAMETERS.

alternates `dictionary` `array` `none`

Dictionary (`((de: "de/contact/index.html"))`) or array of `(hreflang: ..., href: ...)` records.

site-url `str` `none` = none

Optional site origin URL used to expand relative bundle paths into absolute URLs.

tr(from: none, ..branches) -> `content`

Declare inline or block multilingual content (`#tr(en: [Hello], de: [Grüezi])`), or look up all translations for a key from a `@preview/linguify`-compatible `lang.toml` dictionary (`#tr("abstract", from: toml("lang.toml"))`).

Inside each branch, `text.lang` is set to that branch's BCP-47 language code so both Typst's native localized smartquotes/supplements and `@preview/linguify` calls resolve per branch automatically. In HTML export, emits `<span class="thyme-tr" lang="...">` (or `<div class="thyme-tr" lang="...">` when containing block elements) toggled at zero runtime cost via keyboard-accessible CSS radio inputs. In paged PDF export, renders the branch matching active `text.lang` (falling back to the primary declared branch).

```
#tr(en: [Programming languages], de: [Programmiersprachen])
```

```
Programming languages
```

PARAMETERS.

from `dictionary` `none` = none

Optional `@preview/linguify`-compatible dictionary (`toml("lang.toml")`) when looking up a string key.

..branches `any`

Positional string key (when `from` is provided) or named language branches (`en: [...], de: [...]`).

i18n-css(langs) -> `str`

Generate the minimal CSS stylesheet rules driving zero-JavaScript, keyboard-accessible `<input type="radio">` language switching for a bilingual/multilingual page's `langs` tuple.

PARAMETERS.

langs `array`

Ordered list of active language codes on the page (e.g. `("en", "de")`).

render-lang-inputs(langs) -> `content`

Emit visually-hidden, keyboard-accessible top-of-`<body>` radio `<input>` controls for zero-JS language switching when `langs` has two or more languages.

lang-switcher(langs) -> `content`

Render the clickable `en · de` `<label>` switcher bar for zero-JS CSS language toggling. Emits nothing when `langs` has fewer than two languages or outside HTML targets.

PARAMETERS.

langs `array`

Language codes (`("en", "de")`).

3.4 Sitemap, `robots.txt` & Feeds (`src/seo.typ`)

```
content-to-feed-html(
  node,
  site-url: none,
  doc-path: none,
  lang: none,
  label-urls: (:),
) -> str
```

Serialize a Typst `content` AST into clean, reader-friendly semantic HTML for full-content Atom (`<content type="html">`) and RSS 2.0 (`<content:encoded>`) feeds.

PARAMETERS.

node `any`
Typst `content` body to serialize.

site-url `str` `none` = none
Canonical site origin URL used to resolve relative links.

doc-path `str` `none` = none
Document bundle path (e.g. `"posts/foo/index.html"`).

lang `str` `none` = none
Preferred language code when flattening `#tr(...)` nodes.

label-urls `dictionary` = (:)
Optional map of bundle `<label>` names (`"about"`, `"rss"`, `"fixed-points"`) to canonical URLs.

```
render-seo-meta(
  title: none,
  site-title: none,
  description: none,
  author: none,
  canonical-url: none,
  created: none,
  modified: none,
  monograph: none,
  tags: (),
  is-article: false,
  lang: "en",
  open-graph: false,
  scholar-meta: false,
) -> content
```

Emit opt-in OpenGraph (`og:*`, `article:*`) and Google Scholar Highwire Press (`citation_*`) `<meta>` tags inside `<head>`.

```
sitemap-xml(
  site-url: "https://example.com",
  entries: (),
  include-root: true,
  root-alternates: none,
) -> str
```

Generate a spec-compliant `sitemap.xml` (`http://www.sitemaps.org/schemas/sitemap/0.9`) via `@preview/exemel` (`sitemap-encode`) covering `index.html`, all public pages/posts, and any `hreflang` alternate URLs.

PARAMETERS.

site-url `str` = "https://example.com"
Canonical site origin URL (`"https://example.com"`).
entries `array` = ()
Public page/post records
(`((html-path: "...", created: ..., modified: ..., alternates?: ...)`).
include-root `bool` = true
Include the site root URL (`/`) as the first `<url>` entry.
root-alternates `dictionary` `array` `none` = none
Root-level `hreflang` alternates (`(("de": "/de/"))`).

```
robots-txt(site-url: "https://example.com", sitemap-path:
"sitemap.xml") -> str
```

Generate a standard `robots.txt` pointing crawlers to `sitemap.xml`.

PARAMETERS.
site-url `str` = "https://example.com"
Canonical site origin URL (`"https://example.com"`).
sitemap-path `str` `none` = "sitemap.xml"
Relative path to the emitted XML sitemap, or `none` to omit the `Sitemap:` line.

```
atom-xml(
  site-title: [Site],
  site-subtitle: none,
  site-url: "https://example.com",
  author: none,
  atom-path: "atom.xml",
  feed-content: "summary",
  feed-epoch: auto,
  label-urls: (:),
  posts: (),
) -> str
```

Generate an Atom 1.0 XML feed string via `@preview/exemel` (`atom-encode`), supporting both summary-only (`feed-content: "summary"`) and full semantic HTML (`feed-content: "full"`) entries.

PARAMETERS.
site-title `content` `str` = [Site]
Feed title.
site-subtitle `content` `str` `none` = none
Optional Atom `<subtitle>`.
site-url `str` = "https://example.com"
Canonical site origin URL.
author `content` `str` `none` = none
Default feed author name.
atom-path `str` = "atom.xml"
Relative path to the Atom feed (`atom.xml`).
feed-content `str` = "summary"
Entry payload mode (`"summary"` or `"full"`).
feed-epoch `auto` `datetime` = auto
Optional RFC 4151 feed `<id>` minting date (`auto` derives from earliest post).
label-urls `dictionary` = (:)
Optional map of bundle `<label>` names to canonical URLs for cross-bundle links.
posts `array` = ()
Page/post records returned by `query-pages()`.

```
rss-xml(
  site-title: [Site],
  site-subtitle: none,
  site-url: "https://example.com",
  language: none,
  rss-path: "rss.xml",
  feed-content: "summary",
  label-urls: (:),
  posts: (),
) -> str
```

Generate an RSS 2.0 XML feed string via `@preview/exemel` (`rss-encode`), supporting both `<description>` summaries (`feed-content: "summary"`) and full `<content:encoded>` HTML (`feed-content: "full"`).

PARAMETERS.

site-title `content` `str` = `[Site]`
Channel `<title>`.

site-subtitle `content` `str` `none` = `none`
Channel `<description>` (falls back to `site-title` when `none`).

site-url `str` = `"https://example.com"`
Canonical site origin URL.

language `str` `none` = `none`
Optional `<language>` tag (e.g. `"en"`).

rss-path `str` = `"rss.xml"`
Relative path to the RSS feed (`rss.xml`) for `atom:link rel="self"`.

feed-content `str` = `"summary"`
Entry payload mode (`"summary"` or `"full"`).

label-urls `dictionary` = `(:)`
Optional map of bundle `<label>` names to canonical URLs for cross-bundle links.

posts `array` = `()`
Page/post records returned by `query-pages()`.

3.5 URLs, Slugs, AST Traversal & Plain Text (`src/paths.typ`, `src/text.typ`, `src/tree.typ`)

```
resolve-abs-length(val, fallback: 0pt) -> length
```

Safely resolve a `length` or `relative` length to an absolute `length`.

PARAMETERS.

val `length` `relative` `auto` `none`
Length value (`length`, `relative`, `auto`, or `none`).

fallback `length` = `0pt`
Fallback length when `val` is `auto` or `none`.

```
slugify(input) -> str
```

Convert a string or content value into a URL-safe kebab-case slug.

PARAMETERS.

input `content` `str`
Title or identifier to slugify.

```
humanize-slug(s) -> str
```

Humanize a kebab-case slug or label name for fallback link display.

join-path(..parts) -> `str`

Join path segments with single `/` separators and no leading or trailing slash.

unwrap-typ-path(p) -> `str` `none`

Extract the normalized project-root path string from a Typst `0.15` `path(...)` value (e.g. `path("posts/impli.typ")` -> `"posts/impli.typ"`), unescaping string escapes.

slug-from-typ-path(raw-path) -> `str` `none`

Derive a URL slug from a `.typ` source path (`"posts/impli.typ"` -> `"impli"`, `"posts/gol/index.typ"` -> `"gol"`).

relative-typ-path(file-path, site-path: none) -> `str` `none`

Strip the directory prefix of `site-src` (`path("site.typ")`) from a mounted `.typ` path so routes stay relative to the site root even when compiled from a parent `--root`.

post-html-path(slug, posts-dir: "posts") -> `str`

Canonical bundle HTML output path for a post (`posts/<slug>/index.html` by default).

route-html-path(rel-typ) -> `str`

Convert a relative `.typ` page path into its canonical bundle HTML path (`"contact.typ"` -> `"contact/index.html"`, `"docs/intro/index.typ"` -> `"docs/intro/index.html"`).

source-path-for(doc-path) -> `str`

Compute the sibling Tadi Web `.typ` bundle path for any HTML document path:

- `"index.html"` -> `"site.typ"`
- `"posts/impli/index.html"` -> `"posts/impli.typ"`
- `"contact/index.html"` -> `"contact.typ"`

PARAMETERS.

doc-path `str`

Bundle HTML document path.

parent-dir(doc-path) -> `str`

Return the parent directory of a bundle document path (e.g. `"posts/gol"`).

slug-from-path(doc-path) -> `str` `none`

Derive a page/post slug from its bundle HTML document path (`"posts/foo/index.html"` -> `"foo"`).

rel-root(doc-path) -> `str`

Compute the relative root prefix (`""`, `"../"`, `"../../"`) for a bundle HTML path.

rel-href(from-doc, to-path) -> `str`

Compute a clean relative href from a bundle HTML document (`from-doc`) to another bundle file (`to-path`).

canonical-page-url(site-url, doc-path) -> `str`

Convert a canonical origin URL + bundle HTML path into a clean public URL (`"posts/foo/index.html"` -> `"https://example.com/posts/foo/"`).

url-authority(url) -> `str`

Extract domain authority from an HTTP(S) URL (`"https://blog.example.org/foo"` -> `"blog.example.org"`).

format-date(d, fmt: "[year]-[month]-[day]") -> `str` `none`

Format a `datetime` or string into ISO `YYYY-MM-DD`.

date-sort-key(d) -> `str`

Sort key for dates (ascending ISO timestamp; reverse for newest-first).

emit-doc-marker(doc-path, doc-label) -> `content`

Emit a zero-footprint `<thyme-doc-marker>` at the start of every bundle `document(...)` so enclosing-document lookup runs in $O(1)$ without cross-document `<thyme-page-meta>` bleed.

find-doc-containing(loc) -> `content` `none`

Find the enclosing `document` element for an arbitrary location in $O(1)$ inside `context`.

find-enclosing-doc(meta-loc, meta-label: <thyme-page-meta>) -> `content` `none`

Find the enclosing `document` element in bundle mode for a metadata or body location.

resolve-bundle-label(lbl) -> `content` `none`

Resolve a `<label>` across the bundle, preferring emitted `document` / `asset` / `heading` targets over mount-marker `metadata` nodes, while falling back to mounted-page `metadata` in paged preview mode so `@slug` still displays the target's real title.

doc-page-meta(doc-el) -> `dictionary` `none`

Look up `<thyme-page-meta>` inside a `document` element if present.

heading-anchor-id(it, auto-slug: true, prior-headings: ()) -> `str` `none`

Derive a deterministic, collision-free HTML `id` attribute for a `heading` element (`str(it.label)` when explicitly labeled, otherwise `slugify(it.body)` with `"section"` fallback and `-2`, `-3`, … suffixing across `prior-headings` in the same document).

PARAMETERS.

it `content`
Target `heading` element.

auto-slug `bool` = true
Generate an automatic slug when `it` has no explicit `<label>`.

prior-headings `array` = ()
Prior `heading` elements within the same document used to deduplicate identical slugs.

```
with-html-headings(
  shift: 0,
  permalinks: false,
  permalink-symbol: "§",
  body,
) -> content
```

Install HTML heading rules mapping Typst headings (`level: 1..6`) to `<h1>`..`<h6>` (with optional `+1` level `shift` when `<header><h1>` already renders the page title), preserving collision-free `id` attributes for `#anchor` links and optional `§` permalinks.

with-smart-refs(ink: luma(18%), muted: luma(52%), body) -> `content`

Install universal cross-bundle `link` and `ref` (`@tag` / `<tag>`) rules, recording outbound page references (`<thyme-outbound-ref>`) for compile-time backlinks and scoping bibliography checks to the current document.

query-backlinks(target-slug, exclude-doc-path: none) -> `array`

Query all public (non-draft, non-unlisted) pages/posts in the bundle that reference `target-slug` via `@slug` or `link(<slug>)`, sorted newest-first. Must be called inside `context`.

PARAMETERS.

target-slug `str`
Target page/post slug or label name (e.g. `"rabbit-holes"`).

exclude-doc-path `str` `none` = none
Optional current document path to exclude self-references.

to-plain-text(it, lang: none) -> `str`

Convert arbitrary Typst content (including `raw`, `math.equation`, and `#tr(...)` bilingual nodes) or primitive values into a plain `str`.

When `#tr(en: [...], de: [...])` nodes are encountered, only the requested `lang` (or first declared language when `lang` is `none`) is extracted so `<title>`, meta descriptions, and RSS / Atom feeds never concatenate translations.

PARAMETERS.

`it` `any`
Value or content tree to flatten into plain UTF-8 text.

`lang` `str` `none` = none
Optional preferred language code when flattening `#tr(...)` nodes.

```
count-words(body) -> int
```

Count prose words in arbitrary Typst content (Unicode-letter aware for English, German umlauts / ß, and accented Latin scripts; counts only the primary branch of `#tr(...)` bilingual blocks and excludes multi-line fenced code blocks).

PARAMETERS.

`body` `content` `str` `none`
Content block to count words in.

```
word-hint(..parts) -> content
```

Emit a zero-footprint `<thyme-word-hint>` marker from inside `context`-wrapped components (`sbox`, `proof`, `sidenote`, `callout`, `epigraph`, `htm`, `h`, etc.) so pages and posts compute full-document prose word counts without double-counting nested components or secondary `#tr(...)` branches.

PARAMETERS.

`..parts` `any`
Content fragments rendered inside a `context` closure.

```
estimate-reading-minutes(words, wpm: 200) -> int
```

Estimate reading time in minutes from a prose word count.

PARAMETERS.

`words` `int`
Total prose word count.

`wpm` `int` = 200
Reading speed in words per minute.

```
query-body-words(base-words, doc-el: none) -> int
```

Sum `base-words` with all `<thyme-word-hint>` markers emitted inside `doc-el` (or inside the body region following `here()` in standalone single-file mode). Must be called inside `context`.

```
short-hash(data) -> str
```

Compute a deterministic 8-hex-character digest over 100% of the bytes in `data` (`str` or `bytes`) via 32-bit word mixing for content-addressed asset cache keys (e.g. `css/style.css?v=8f3a1c02`).

```
find-content(c, match-fn, descend: _ => true) -> any
```

Find the first non-`none` result produced by `match-fn(node)` in pre-order traversal. Set `descend` to prune subtrees (e.g. skipping mounted subpages).

PARAMETERS.

c `any`
Root `content` value (or arbitrary value).

match-fn `function`
Visitor returning `none` to continue searching or any non-`none` value to short-circuit.

descend `function` = `_ => true`
Predicate deciding whether to recurse into `c`'s children.

```
any-content(c, pred, descend: _ => true) -> bool
```

Return `true` when any node in `c` satisfies `pred(node)`.

```
collect-content(c, extract-fn, descend: _ => true) -> array
```

Collect an array of items extracted by `extract-fn(node)` across `c`. `extract-fn(node)` should return an `array` of records (`()` when nothing matches).

PARAMETERS.

c `any`
Root `content` value.

extract-fn `function`
Extractor returning `()` or an `array` of extracted items for `node`.

descend `function` = `_ => true`
Predicate deciding whether to recurse into `c`'s children.

```
map-containers(c, transform-fn) -> content
```

Transform nodes inside `c` via `transform-fn(node)` (which returns `auto` to descend, or a replacement `content` value), preserving sequences, HTML-safe layout containers (`block`, `box`, `align`, `pad`, `list.item`, `enum.item`, `quote`), and typed `html.*` (`html.elem`) wrappers at arbitrary nesting depth while short-circuiting untouched subtrees.

PARAMETERS.

c `any`
Root `content` tree.

transform-fn `function`
Mapper returning `auto` when `c` is not replaced directly, or replacement `content`.

04 `blog` Preset Reference (`src/blog/*.typ`)

4.1 Blog Bundler (`src/blog/site.typ`)

```
post-list(show-summary: true, tag: none, lang: none) -> content
```

Render a newest-first `<ul class="entries">` list of all public posts in the bundle.

PARAMETERS.

`show-summary` `bool` = `true`
Include each post's `summary` (or `subtitle` fallback) beneath its title row.

`tag` `str` `none` = `none`
Optional tag filter.

`lang` `str` `none` = `none`
Optional primary language filter.

```
tag-cloud(tags-dir: "tags") -> content
```

Render a monochrome `#tag` index list (`<ul class="tag-cloud">`) with post counts, linking to `/tags/<slug>/index.html` inside bundles.

PARAMETERS.

`tags-dir` `str` = `"tags"`
Output directory prefix for tag archive pages.

```
site(
  title: auto,
  description: none,
  url: "https://example.com",
  author: none,
  lang: "en",
  alternates: none,
  posts-dir: "posts",
  tag-pages: true,
  tags-dir: "tags",
  src: none,
  expose-source: true,
  stylesheets: (),
  favicon: none,
  pages: (),
  assets: (),
  not-found-page: true,
  sitemap-path: "sitemap.xml",
  robots: auto,
  rss-path: "rss.xml",
  atom-path: "atom.xml",
  feed-content: "summary",
  feed-epoch: auto,
  heading-permalinks: false,
  backlinks: false,
  post-nav: false,
  copy-button: auto,
  open-graph: false,
  scholar-meta: false,
  head: none,
  theme: (:),
  body,
) -> content
```

Scholarly monochrome blog bundler (`#show: site.with(...)` after `#import blog: *`, or `#show: blog.site.with(...)`).

Renders `site.typ`'s body as `index.html` (`<site>`), emits every mounted `#post(path("posts/foo.typ")) <foo>` as `<posts-dir>/foo/index.html` (`<foo>`) and any `#mount(path("about.typ")) <about>` subpages, generates static `/tags/` archive pages, co-emits sibling Tadi Web `.typ` plaintext sources, and builds content-hashed `css/style.css` (`<stylesheet>`), `404.html`, `sitemap.xml` (`<sitemap>`), `robots.txt`, `rss.xml` (`<rss>`), and `atom.xml` (`<atom>`).

PARAMETERS.

`title` `auto` `content` `str` = auto
Site title (`auto` infers from the first `= ...` heading in `body`, or pass any `str` / rich `content`).

`description` `content` `str` `none` = none
Short site description for `<meta name="description">` and feed subtitles.

`url` `str` = "https://example.com"
Canonical origin URL (without trailing slash) for canonical links, `sitemap.xml`, and feeds.

`author` `content` `str` `none` = none
Default author inherited by posts that omit `author` and emitted in Atom `<author>`.

`lang` `str` `array` = "en"
Default site language (`"en"` or `("en", "de")`).

`alternates` `dictionary` `array` `none` = none
Optional `hreflang` alternate URLs for the site root (`(de: "/de/")`).

`posts-dir` `str` = "posts"

Output directory prefix for emitted post pages (`posts/<slug>/index.html`).

`tag-pages` `bool` = `true`

Generate static `/tags/index.html` and `/tags/<tag>/index.html` archive pages and link `#tag` badges.

`tags-dir` `str` = `"tags"`

Output directory prefix for static tag archive pages (`tags/<tag>/index.html`).

`src` `path` `str` `none` = `none`

Capability `path("site.typ")` (or `str`) of the root entrypoint so Tadi Web source view emits `/site.typ`.

`expose-source` `bool` `str` = `true`

Co-emit sibling `.typ` plaintext source files (`/site.typ`, `/posts/<slug>.typ`) and render a top-right `.typ` link (`true` uses `".typ"`, `false` disables, or pass custom label `str`).

`stylesheets` `array` = `()`

Additional user stylesheets (`path("extra.css")`) appended after `css/style.css`.

`favicon` `path` `dictionary` `none` = `none`

Optional site favicon (`path("favicon.svg")` or `(path: "favicon.svg", data: ...)`).

`pages` `array` = `()`

Additional standalone HTML pages `(slug: "about", title: [About], body: [...])`.

`assets` `array` = `()`

Site-wide static assets `(path: "pgp.txt", data: ..., label: none)`.

`not-found-page` `bool` `content` = `true`

Emit `404.html` styled with the active monochrome palette and typography.

`sitemap-path` `str` `none` = `"sitemap.xml"`

Output bundle path for the XML sitemap (`"sitemap.xml"`), or `none` to disable.

`robots` `auto` `bool` = `auto`

Emit `robots.txt` pointing to `sitemap-path` (`auto` enables whenever `sitemap-path != none`).

`rss-path` `str` `none` = `"rss.xml"`

Output bundle path for the RSS 2.0 feed, or `none` to disable RSS generation.

`atom-path` `str` `none` = `"atom.xml"`

Output bundle path for the Atom 1.0 feed, or `none` to disable Atom generation.

`feed-content` `str` = `"summary"`

Feed payload mode (`"summary"` for compact descriptions or `"full"` for full semantic HTML).

`feed-epoch` `auto` `datetime` = `auto`

Optional RFC 4151 feed `<id>` minting date (`auto` derives from earliest post).

`heading-permalinks` `bool` = `false`

Append `§` hover permalinks on section headings (`h2`–`h6`) across posts and pages.

`backlinks` `bool` = `false`

Render compile-time backlinks footers across posts and pages by default.

`post-nav` `bool` = `false`

Render chronological older / newer navigation links at the bottom of posts by default.

`copy-button` `auto` `bool` = `auto`

Inject copy-button JavaScript (`auto` injects only on pages containing `#zebraw` code blocks).

`open-graph` `bool` = `false`

Emit OpenGraph `<meta property="og:*">` tags across pages and posts by default.

`scholar-meta` `bool` = `false`

Emit Google Scholar Highwire `<meta name="citation_*">` tags on posts by default.

`head` `content` `none` = `none`

Additional `<head>` elements injected on `index.html`.

`theme` `dictionary` = `(:)`

Theme overrides merged onto `default-theme` across both HTML CSS and paged previews.

`body` `content`

Homepage markup (`= Heading`, `#links(direction: "row", ...)`, and mounted `#post(path("...")) <slug>` entries).

4.2 Post Template (`src/blog/post.typ`)

`post-nav`(older-label: "← older", newer-label: "newer →") -> `content`

Query all public (non-draft, non-unlisted) posts in the active bundle, sorted newest-first. Must be called inside `context`. -> array Render chronological older / newer navigation links at the bottom of a post. Emits nothing when outside a bundle or when only one public post exists.

PARAMETERS.

older-label `content` `str` = "← older"
Label shown above the chronologically older post.

newer-label `content` `str` = "newer →"
Label shown above the chronologically newer post.

```
post(
  title: auto,
  subtitle: none,
  created: none,
  modified: none,
  date: none,
  updated: none,
  lang: auto,
  alternates: none,
  author: none,
  monograph: none,
  tags: (),
  summary: none,
  assets: (),
  slug: auto,
  path: none,
  src: none,
  home-text: "/..",
  wpm: 200,
  show-wordcount: true,
  draft: false,
  unlisted: false,
  numbered-headings: true,
  heading-permalinks: auto,
  footnotes-as-sidenotes: true,
  citations-as-sidenotes: false,
  show-backlinks: auto,
  show-post-nav: auto,
  copy-button: auto,
  open-graph: auto,
  scholar-meta: auto,
  head: none,
  theme: (:),
  page-width: auto,
  margin: auto,
  body,
) -> content
```

Dual-purpose scholarly post template and inline capability-path mounter:

- In a post file (`posts/foo.typ` or `posts/foo/site.typ`): `#show: post.with(title: [...], author: "...", created: ..., modified: ...)`
- In `site.typ`: `#post(path("posts/foo.typ")) <foo>` (evaluates the post AND exposes `/posts/foo.typ` as Tadi Web plaintext source; `include "posts/foo.typ"` is also accepted).

PARAMETERS.

title `auto` `content` `str` = auto

Post title (`<h1>` in HTML, semibold heading in paged mode); `auto` detects a mounted post in `site.typ`.

`subtitle` `content` `str` `none` = none

Optional italic subtitle beneath `title` (also used as fallback summary).

`created` `datetime` `str` `none` = none

Creation date (`YYYY-MM-DD`); drives feed publication timestamps and newest-first sort order.

`modified` `datetime` `str` `none` = none

Last-modified date (`YYYY-MM-DD`); drives `sitemap.xml` `<lastmod>` and Atom `<updated>`.

`date` `datetime` `str` `none` = none

Alias for `created`.

`updated` `datetime` `str` `none` = none

Alias for `modified`.

`lang` `auto` `str` `array` = auto

Post language (`auto` inherits from `site` and auto-detects `#tr(...)` bilingual blocks; `"en"`, `"de"`, or `("en", "de")`).

`alternates` `dictionary` `array` `none` = none

Optional `hreflang` alternate URLs (`(de: "de/posts/foo/index.html")`).

`author` `content` `str` `none` = none

Post author; falls back to `site.with(author: ...)` inside a bundle.

`monograph` `content` `str` `none` = none

Optional EWD-style monograph code (e.g. `"EWD-02"`) rendered right beneath `author` in post frontmatter.

`tags` `array` = ()

Plain `#tag` strings rendered in `Commit Mono` and exported as RSS / Atom `<category>`.

`summary` `content` `str` `none` = none

Short abstract displayed on homepage entry cards and in RSS / Atom feeds.

`assets` `dictionary` `array` = ()

Co-located files (`("dfa.svg": path("dfa.svg"))`) emitted into `posts/<slug>/`.

`slug` `auto` `str` = auto

Explicit slug override; defaults to `<label>` or `.typ` filename.

`path` `str` `none` = none

Optional custom bundle output path when mounting in `site.typ` (defaults to `<posts-dir>/<slug>/index.html`).

`src` `path` `str` `none` = none

Optional `.typ` source override when mounting via `include` instead of `path(...)`.

`home-text` `content` `str` `none` = "/.."

Backlink label at the top-left of HTML post pages (automatically omitted in PDF and `@media print`).

`wpm` `int` = 200

Words-per-minute reading speed used to compute `reading` minutes.

`show-wordcount` `bool` = true

Display `words` and `reading` rows in the post frontmatter block.

`draft` `bool` = false

Exclude post from homepage listings, `sitemap.xml`, and RSS / Atom feeds when `true`.

`unlisted` `bool` = false

Emit the HTML page at its URL, but omit it from homepage listings, `sitemap.xml`, and feeds.

`numbered-headings` `bool` = true

Prefix top-level `=` sections with `01`, `02`, … in `Commit Mono`.

`heading-permalinks` `auto` `bool` = auto

Append `§` hover permalinks on section headings (`auto` inherits from `site`).

`footnotes-as-sidenotes` `bool` = true

Automatically route standard `#footnote[...]` calls into the right margin.

`citations-as-sidenotes` `bool` = false

Automatically route normal `@bib` bibliography citations into right-margin sidenotes.

`show-backlinks` `auto` `bool` = auto

Render compile-time backlinks footer at the bottom of the post (`auto` inherits from `site`).

`show-post-nav` `auto` `bool` = auto

Render chronological older / newer navigation at the bottom of the post (`auto` inherits from `site`).

copy-button `auto` `bool` = auto
Inject copy-button JavaScript (`auto` injects only when the post contains `#zebraw` code blocks).
open-graph `auto` `bool` = auto
Emit OpenGraph `<meta property="og:*">` tags (`auto` inherits from `site`).
scholar-meta `auto` `bool` = auto
Emit Google Scholar Highwire `<meta name="citation_*">` tags (`auto` inherits from `site`).
head `content` `none` = none
Additional `<head>` elements injected into this post's HTML document.
theme `dictionary` = (:)
Per-post theme overrides layered onto the active site theme.
page-width `auto` `length` = auto
Paged preview width override (`auto` uses `theme.page-width`).
margin `auto` `dictionary` = auto
Paged preview margin override (`auto` uses `theme.margin`).
body `path` `content`
Post body markup (when used via `#show: post.with(...)`) or `path("posts/...")` / `include "posts/..."` (when mounted in `site.typ`).

post-asset `content`

Declare a co-located file asset inline inside a post body (`#post-asset("dfa.svg", path("dfa.svg")) <dfa>` or `read("dfa.svg", encoding: none)`).

4.3 Scholarly Environments & Components (`src/blog/components.typ`)

grid-table(body, ..rest) -> `content`

Inner crosshair grid table built on `tablem` (`left` & `top` hairlines between cells).

PARAMETERS.
body `content`
Markdown-style pipe table content block.
..rest `any`
Additional arguments forwarded to `tablem`.

```
zebraw(
  ..args,
  highlight-color: auto,
  background-color: auto,
  comment-color: auto,
  lang-color: auto,
) -> content
```

Monochrome-framed code block wrapper around `@preview/zebraw:0.6.3` with native syntax highlighting, line numbers, line highlights, and HTML copy button.

PARAMETERS.
..args `any`
Code block and options forwarded to `@preview/zebraw`.
highlight-color `auto` `color` = auto
Highlighted line background (`auto` uses `theme.surface-2-light`).
background-color `auto` `color` = auto
Code block background (`auto` uses `theme.surface-1-light`).
comment-color `auto` `color` = auto
Line-comment annotation fill (`auto` uses `theme.surface-2-light`).

lang-color `auto` `color` = auto
Language badge fill (`auto` uses `theme.surface-2-light`).

```
sbox(
  title,
  ..sections,
  variant: "recipe",
  kind: none,
) -> content
```

Multi-section monochrome environment box with uppercase sans-serif `kind` label. Rendered as `<div class="blog-sbox">` in HTML so browser Reader Modes preserve primary theorems, lemmas, definitions, and recipes verbatim.

PARAMETERS.
title `content` `str` `none`
Box title (`none` or `[]` omits the underlined title).
..sections `content`
One or more body content blocks separated by dotted hairline rules.
variant `str` = "recipe"
Visual surface variant (`"definition"`, `"theorem"`, `"example"`, `"recipe"`).
kind `str` `none` = none
Uppercase sans-serif category label (e.g. `"Definition"`, `"Theorem"`).

```
definition(title, ..sections) -> content
```

Mathematical definition environment (`DEFINITION.`).

```
#definition[Pointed CPO][
  An $omega$-CPO $(D, subset.eq.sq)$ with least element $bot in D$.
]
```

DEFINITION. Pointed CPO
An ω-CPO $(D, \sqsubseteq)$ with least element $\bot \in D$.

PARAMETERS.
title `content` `str` `none`
Definition title.
..sections `content`
One or more body sections.

```
theorem(title, ..sections) -> content
```

Theorem environment (`THEOREM.`).

```
#theorem[Kleene Fixed Point][
  Every Scott-continuous map $F : D -> D$ has $"lfp"(F) = union.sq.big_(n >= 0) F^n (bot)$.
]
```

THEOREM. Kleene Fixed Point
Every Scott-continuous map $F : D \to D$ has $\mathrm{lfp}(F) = \bigsqcup_{n \geq 0} F^n(\bot)$.

PARAMETERS.
title `content` `str` `none`
Theorem title.
..sections `content`
One or more body sections.

```
lemma(title, ..sections) -> content
```

Lemma environment (`LEMMA.`).

proposition(title, ..sections) -> content

Proposition environment (PROPOSITION.).

corollary(title, ..sections) -> content

Corollary environment (COROLLARY.).

example(title, ..sections) -> content

Example environment (EXAMPLE.).

recipe(title, ..sections) -> content

Recipe / procedure environment (RECIPE.).

remark(title, ..sections) -> content

Remark environment (REMARK.).

proof(title: [Proof], body) -> content

Proof block with subtle left rule and flush-right hollow Halmos square (sym.square.stroked).

```
#proof[
  By induction on $n$, $F^n (bot) subset.eq.sq e$ for every pre-fixed point $e$.
]
```

PROOF
By induction on n, $F^n(\bot) \sqsubseteq e$ for every pre-fixed point e. □

PARAMETERS.

title content str = [Proof]
Uppercase sans-serif lead-in label (e.g. [Proof] or [Proof sketch]).

body content
Proof body.

callout(kind: "note", title: auto, body) -> content

Monochrome left-bordered callout (note, tip, warning, important). Rendered as in HTML so Reader Modes preserve important/warning callouts inline.

PARAMETERS.

kind str = "note"
Callout kind ("note", "tip", "warning", "important").

title auto content str none = auto
Custom heading (auto uses kind, none omits heading).

body content
Callout content.

epigraph(source: none, body) -> content

Section-opening italic epigraph with optional right-aligned attribution.

```
#epigraph(source: [Edsger W. Dijkstra, EWD249])[
  Simplicity is prerequisite for reliability.
]
```

Simplicity is prerequisite for reliability.

\- Edsger W. Dijkstra, EWD249

PARAMETERS.

source content str none = none

Optional attribution rendered flush-right in Commit Mono.

body content

Quotation body.

wide(body) -> content

Full-width container spanning both the prose column and the right sidenote margin. Automatically clears any active right-margin sidenotes in both HTML (clear: right) and paged mode (@preview/marge page state).

PARAMETERS.

body content

Wide table, figure, or listing body.

diagram(caption: none, invert-dark: true, body) -> content

Render arbitrary Typst graphics natively in paged previews and as inline <svg> in HTML (automatically inverted in dark mode unless invert-dark: false).

PARAMETERS.

caption content str none = none

Optional figure caption.

invert-dark bool = true

Invert monochrome line-art <svg> automatically in dark mode (false preserves dark-native or multi-hue graphics).

body content

Typst drawing or diagram content.

kbd(key) -> content

Monochrome keyboard shortcut badge (#kbd("Ctrl+P")).

```
Press #kbd("Ctrl+P") for archival two-column print output.
```

Press Ctrl+P for archival two-column print output.

PARAMETERS.

key content str

Key label text.

links(direction: "row", ..items) -> content

General links layout (direction: "row" for horizontal, direction: "col" for vertical).

PARAMETERS.

direction str = "row"

Direction of layout ("row" for horizontal, "col" for vertical).

..items content

Link elements (e.g. link("https://example.org")[example.org], [@rss[rss]]).

```
toc(title: none, depth: 2, numbered: true) -> content
```

Render a monochrome table of contents (`<nav class="blog-toc">` in HTML, hairline-bordered outline in paged PDF) scoped to the enclosing page or post.

PARAMETERS.

title `content` `str` `none` = none
Optional uppercase heading above the outline (`none` omits heading).

depth `int` = 2
Maximum heading depth to include (`1` = top-level `=` sections, `2` = `=` + `==`, etc.).

numbered `bool` = true
Prefix top-level `=` entries with `01`, `02`, … in `Commit Mono`.

```
tablem function
```

Markdown-like pipe table (`@preview/tablem:0.3.0`).

```
three-line-table function
```

Classic academic three-line table (`@preview/tablem:0.3.0`).

```
codeblock function
```

Alias for `zebraw`.

```
hr content
```

Minimalist horizontal rule (`<hr>` in HTML, hairline rule in paged previews).

4.4 Right-Margin Sidenotes (`src/blog/notes.typ`)

```
sidenote(
  dy: 0pt,
  gap: 0.55em,
  padding: (left: 1.4em, right: 1.8cm),
  numbered: true,
  body,
) -> content
```

Place a numbered (or unnumbered) note in the right margin (`@preview/marge` in paged previews; `<span class="sidenote">` float on desktop HTML and `@media print`, collapsing into a tap- or keyboard-expandable numbered card on viewports `<= 1100px`).

PARAMETERS.

dy `length` `ratio` `relative` = 0pt
Vertical offset adjustment in paged mode.

gap `length` = 0.55em
Minimum vertical separation between consecutive paged margin notes.

padding `length` `dictionary` = (left: 1.4em, right: 1.8cm)
Horizontal inset inside the right margin column in paged mode.

numbered `bool` = true
Display synchronized superscript numbers in prose and at the start of the margin note.

body `content`
Margin note content.

marginnote(dy: 0pt, gap: 0.55em, body) -> content

Place an unnumbered margin note alongside the current paragraph ([+] toggle on <= 1100px).

PARAMETERS.

dy length ratio relative = 0pt
Vertical offset adjustment in paged mode.

gap length = 0.55em
Minimum vertical separation between consecutive paged margin notes.

body content
Margin note content.

```
mcite(
  key,
  supplement: none,
  inline: false,
  dy: 0pt,
) -> content
```

Cite a bibliography entry (label or str key) while placing its full formatted bibliographic entry directly into a right-margin sidenote alongside the paragraph.

PARAMETERS.

key label str
Bibliography key (<knuth84> or "knuth84").

supplement content none = none
Optional page/section supplement (e.g. [p. 42]).

inline bool = false
Emit an inline short citation ([1] / (Knuth, 1984)) before the margin note (false uses only the numbered sidenote superscript).

dy length ratio relative = 0pt
Vertical offset adjustment in paged mode.

sn function

Short alias for sidenote.

mn function

Short alias for marginnote.

4.5 Monochrome Theme & Fonts (src/blog/theme.typ)

resolve-theme(overrides) -> dictionary

Merge a user theme override dictionary onto default-theme.

PARAMETERS.

overrides dictionary none
Partial theme dictionary overriding any subset of default-theme keys.

active-theme() -> dictionary

Resolve the active theme inside `context` (preferring any enclosing per-post `<blog-post-theme>` override over the site-wide `<site-config>`).

stylesheet-css(theme: default-theme) -> str

Render `src/blog/style.css` substituting all `oklch(...)` palette tokens, `--serif` / `--math` / `--sans` / `--mono` font stacks, and `@font-face` declarations from `theme`.

PARAMETERS.

theme dictionary = default-theme
Theme dictionary (`default-theme` or user overrides).

default-woff2-files array

Canonical `.woff2` filenames expected under `theme.font-base` when self-hosting upstream *STIX Two* (`v2.13b171`) and *Commit Mono* (`v1.143`) web fonts.

default-theme dictionary

Default perceptual `oklch(L, 0, 0deg)` monochrome palette, unified *STIX Two* + *Helvetica Neue* + *Commit Mono* font stacks, and paged preview geometry.